

Compressió de dades i imatges - GEI - FIB

Tema 5: Altres tècniques

Anna de Mier, Lluís Vena
anna.de.mier@upc.edu, lluis.vena@upc.edu

Departament de Matemàtiques • Universitat Politècnica de Catalunya

Març 2026

Capítol 6 de [Introduction to Data Compression](#) Sayood, Khalid Morgan
Kaufmann, 2012, 4th ed.

- 1 Burrows-Wheeler (BW)
- 2 Move to Front (MtF)
- 3 Codis de Rice
- 4 Prediction with Partial Match (PPM)

Burrows-Wheeler (BW)

Aprofita el context per agrupar caràcters iguals.

Exemple: cadacaapasta

Permutacions cícliques: Ordenació lexicogràfica:

cadacaapasta	0 aacpastacadac
acadacaapast	1 acaacpastacad
tacadacaapas	2 acadacaapast
stacadacaapa	3 acapastacadaca
astacadacaap	4 adacaacpastac
pastacadacaac	5 apastacadacaac
apastacadacaac	6 astacadacaap
capastacadaca	7 caacapastacada
acapastacadaca	8 cadacaapasta
aacpastacadac	9 capastacadaca
caacpastacada	10 dacaapastaca
acaacpastacad	11 pastacadacaaca
dacaacpastaca	12 stacadacaapaca
adacaacpastac	13 tacadacaapas

El resultat de la transformació és l'última columna de l'ordenació lexicogràfica i la posició que ocupa el missatge original en aquesta ordenació lexicogràfica: cdtaccpaaaaaas, 8

Burrows-Wheeler (BW)

Codi: cdtaccpaaaaaas, 8

Ordenant alfabèticament, sabem les primeres lletres, a més de les últimes de totes les permutacions.

L'última lletra del missatge la sabem perquè ens diuen la posició del missatge original a la llista de permutacions ordenades, en aquest cas **a**.

Per saber la penúltima lletra ens fixem que si llegim des de l'última lletra, aleshores la penúltima lletra passarà a ocupar l'última posició i serà una de les que hi ha a l'última posició de les paraules que comencen per **a**.

Quina? doncs com que estan ordenades alfabèticament si la **a** ocupa la 3a posició entre les **a** en fer el desplaçament cíclic continuarà ocupant la 3a posició entre les **a**, per tant la penúltima lletra serà **t**. Per saber la prèvia a la **t**, fem el mateix, si eliminem la **t** i la col·loquem a l'inici ara l'antepenúltima lletra passarà a ocupar l'última posició i serà **s**.

L'anterior a la **s** serà la setena **a**.

L'anterior a la (setena) **a** serà la **p**.

L'anterior a la **p** serà la sisena **a**...

Codi: cdtaccpaaaaaas, 8

```
a.....c 0
a.....d 1
a.....t 2
a.....a 3
a.....c 4
a.....c 5
a.....p 6
c.....a 7
c.....a <-----
c.....a 9
d.....a 10
p.....a 11
s.....a 12
t.....s 13
```

Missatge reconstruït (encara que també sabem la primera lletra):a

Iter: 1	Iter: 2	Iter: 3	Iter: 4	...	Iter: 13
a.....c	a.....c	a.....c	a.....c		a.....c
a.....d	a.....d	a.....d	a.....d		a.....d
a---->t	a.....t	a.....t	a.....t		a.....t
a.....a	a.....a	a.....a	a.....a		a.....a
a.....c	a.....c	a.....c	a.....c		a---->c
a.....c	a.....c	a.....c	a.....c		a.....c
a.....p	a.....p	a.....p	a---->p		a.....p
c.....a	c.....a	c.....a	c.....a		c.....a
c.....a	c.....a	c.....a	c.....a		c.....a
c.....a	c.....a	c.....a	c.....a		c.....a
d.....a	d.....a	d.....a	d.....a		d.....a
p.....a	p.....a	p.....a	p.....a		p.....a
s.....a	s.....a	s---->a	s.....a		s.....a
t.....s	t---->s	t.....s	t.....s		t.....s

Missatge:

...ta ...sta ...asta ...pasta cadacaacapasta

Burrows-Wheeler (BW)

Observacions:

- També podem recuperar el text original reconstruint des de l'inici cap al final.
- A l'hora de codificar no cal tenir en memòria totes les permutacions, ja que aquestes són cícliques, es pot ordenar alfabèticament directament.
- Usat en Bioinformàtica per trobar *matches* en seqüències d'ADN

Move to Front (MtF)

Codifiquem cada símbol amb la posició que té actualment a la llista, i immediatament després el movem al principi de la llista. Encara que el resultat prengui valors enters en un rang $[0, \text{len}(\text{alfabet})-1]$, estan esbiaixats cap als més petits.

Exemple: paula passa del pau

paula passa del pau	[-, a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, ...]	16
p aula passa del pau	[p, -, a, b, c, d, e, f, g, h, i, j, k, l, n, o, q, ...]	2
pa ula passa del pau	[a, p, -, b, c, d, e, f, g, h, j, k, l, n, o, p, ...]	21
pau la passa del pau	[u, a, p, -, b, c, d, e, f, g, h, j, k, l, n, o, p, ...]	13
paul a passa del pau	[l, u, a, p, -, b, c, d, e, f, g, h, j, k, n, o, p, ...]	2
paula passa del pau	[a, l, u, p, -, b, c, d, e, f, g, h, j, k, n, o, p, ...]	4
paula passa del pau	[-, a, l, u, p, b, c, d, e, f, g, h, j, k, n, o, p, ...]	4
paula p assa del pau	[p, -, a, l, u, b, c, d, e, f, g, h, j, k, n, o, p, ...]	2
paula pa ssa del pau	[a,p, -, l, u, b, c, d, e, f, g, h, j, k, n, o, p, ...]	20
paula pas sa del pau	[s, a,p, -, l, u, b, c, d, e, f, g, h, j, k, n, o, p, ...]	0

Move to Front (MtF)

Exemple: paula passa del pau

paula pass|a del pau [s, a, p, -, l, u, b, c, d, e, f, g, h, j, k, n, o, p, ...] 1
paula passa| del pau [a, s, p, -, l, u, b, c, d, e, f, g, h, j, k, n, o, p, ...] 3
paula passa |del pau [-, a, s, p, l, u, b, c, d, e, f, g, h, j, k, n, o, p, ...] 8
paula passa d|el pau [d, -, a, s, p, l, u, b, c, e, f, g, h, j, k, n, o, p, ...] 9
paula passa de|l pau [e, d, -, a, s, p, l, u, b, c, e, f, g, h, j, k, n, o, p, ...] 6
paula passa del| pau [l, e, d, -, a, s, p, u, b, c, e, f, g, h, j, k, n, o, p, ...] 3
paula passa del |pau [-, l, e, d, a, s, p, u, b, c, e, f, g, h, j, k, n, o, p, ...] 6
paula passa del p|au [p, -, l, e, d, a, s, u, b, c, e, f, g, h, j, k, n, o, p, ...] 5
paula passa del pa|u [a, p, -, l, e, d, s, u, b, c, e, f, g, h, j, k, n, o, p, ...] 7

Codi: [16, 2, 21, 13, 2, 4, 4, 2, 20, 0, 1, 3, 8, 9, 6, 3, 6, 5, 7]

Codis de Rice

S'usen per codificar enters de longitud variable.

Depenen d'un paràmetre k . Sigui $m = 2^k$. Per codificar un enter x :

- 1 Sigui $q = x/m$, (quocient), escriure q 1's.
- 2 Escriure un 0.
- 3 Escriure $x \bmod m$ (residu) amb exactament k bits.

$k = 2, m = 2^2$ y $k = 4, m = 2^4$

x	$k = 2, m = 2^2$	$k = 4, m = 2^4$
0	0 00	0 0000
1	0 01	0 0001
2	0 10	0 0010
3	0 11	0 0011
4	10 00	0 0100
5	10 01	0 0101
6	10 10	0 0110
7	10 11	0 0111
8	110 00	0 1000
...	...	...
14	1110 10	0 1110
15	1110 11	0 1111
16	11110 00	10 0000
17	11110 01	10 0001

El fet d'operar mòdul una potència de 2 fa que sigui ràpid codificar o descodificar.

Prediction with Partial Match (PPM)

Aprofita el context per assignar probabilitats als símbols.

En llegir un nou símbol, l'algorisme de codificació $\mathbf{s}$ es fixa en el context $\mathbf{C}$ d'ordre N anterior (N símbols anteriors) i, basant-se en les dades llegides anteriorment, assigna una probabilitat p que $\mathbf{s}$ segueixi a $\mathbf{C}$. Si la combinació $\mathbf{C} \mathbf{s}$ no ha aparegut fins ara, s'envia un símbol $\mathbf{esc}$ i es mira el context d'ordre $N - 1$, i així successivament.*

Amb aquestes probabilitats s'invoca un Huffman o una codificació aritmètica (dinàmics).

*El context d'ordre 0 de $\mathbf{s}$ seria la freqüència de $\mathbf{s}$, i el d'ordre -1 seria $1/\text{len}(\text{alfabet})$

Prediction with Partial Match (PPM)

Exemple

Suposem que el context d'ordre 3 és la cadena **QUE** (els últims 3 caràcters llegits) que ha aparegut amb anterioritat 27 vegades i que s'ha vist seguit:

- 11 vegades de la lletra **␣** (espai)
- 9 vegades de la lletra **S**
- 6 vegades de la lletra **N**
- 1 vegades de la lletra **M**

Si la següent lletra fos un **␣** s'envia a codificar la lletra **␣** (Huffman o aritmètica) amb la distribució de probabilitats **␣** 11/27, **S** 9/27, **N** 6/27, **M** 1/27 i s'incrementen els contextos de:

- **QUE**+**␣** a 12
- **UE**+**␣** en +1
- **E**+**␣** en +1
- **␣** en +1

Prediction with Partial Match (PPM)

Exemple (cont.)

Si la següent lletra hagués estat **A**, com que no apareix al context **QUE** es canvia el context a **UE**. Si en aquest context tenim **UE**

- +A 26
- +□ 11 (com a mínim)
- ...
-

amb un total de 54 aparicions, s'envia a codificar la lletra **A** amb la distribució de probabilitats $A \ 26/54$, $\square \ 11/54, \dots$ i s'incrementen els contextos

- QUE+A a 1
- UE+A a 27
- E+A en +1
- A en +1